BIG DATA ANALYTICS

Real-Time Wildfire Risk Prediction System using Big Data Analytics

FINAL REVIEW

Faculty: Dr. Sreeja B P

Amitha Mahesh (CB.AI.U4AID24105) | Meghana Reddy (CB.AI.U4AID24131)
Nitheka A (CB.AI.U4AID24135) | Shivvanii S (CB.AI.U4AID24146)

Problem Statement and Objectives

Problem

- Wildfires spread rapidly in hot, dry and windy conditions.
- Weather-only alerts cause false alarms.
- Social-media-only detection is noisy and poorly geotagged.
- Real-world feeds often contain missing readings.

Objectives

1. Store 22.1 GB weather data in replicated HDFS.
2. Clean data and train risk models using Spark MLlib.
3. Generate verified live alerts using weather and social evidence.
4. Display alerts on a real-time map dashboard.

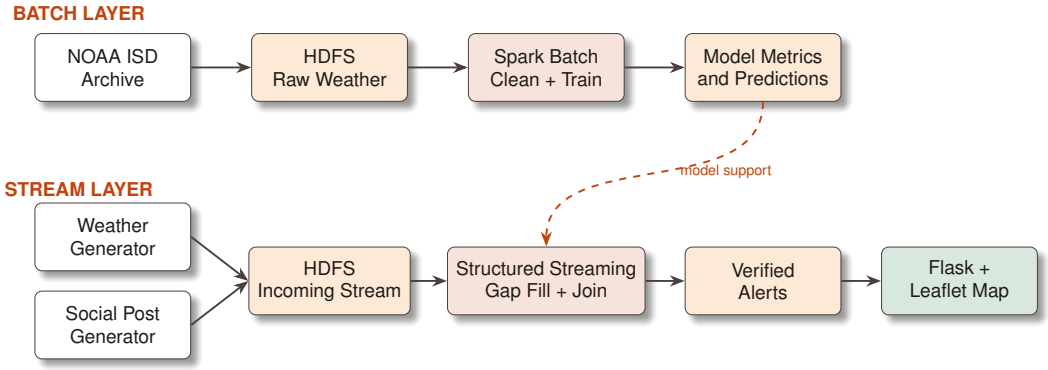
Why Big Data? Why do we need this much data?

- Fire risk depends on temperature and wind, which vary across seasons, regions and extreme-weather days. Years of multi-station data cover all these conditions.
- More data makes the learned risk pattern reliable instead of reflecting a few days or places.
- The archive is too large for a normal single-machine, in-memory script, so we need HDFS storage and parallel Spark processing.
- Live weather and posts arrive continuously, and a warning is useful only within minutes, so streaming is needed.
- Real feeds have missing values and noisy posts; large data and two independent sources reduce the effect of any single bad reading.

Dataset: NOAA Archive and Live Streams

	Historical data: NOAA ISD (batch)	Live data: weather and social posts (stream)
Source	NOAA NCEI Integrated Surface Database, public archive [1]	Simulated by Python weather and social-post generators, written to HDFS `/incoming`
Size	**22.1 GB**, **53,174** gzip station-year files	Continuous small JSON files, growing while the job runs
Volume	Years of observations from many stations; too large for single-machine in-memory processing	One reading every 5 s per station and one post every 3 s, accumulating continuously
Variety	Fixed-width text (structured)	JSON weather readings (structured) and free-text social posts (unstructured)
Velocity	Static archive, processed as a batch	Real-time micro-batches, processed as data arrives
Key attributes	Station id, date, latitude, longitude, temperature, wind speed, dew point	**Weather:** station, lat/lon, temperature, wind, humidity, time. **Social:** post id, text, lat/lon, fire flag, time

System Architecture



Deployment

HDFS: 1 NameNode + 3 DataNodes, replication factor 3 • Spark 3.5.0, Scala 2.12, Java 11 • Flask API: `/api/alerts`, `/api/model`

Spark Workflow

Execution Model

- DataFrame and Spark SQL with Catalyst optimization.
- RDDs used underneath for fixed-width parsing.
- Lazy evaluation builds a DAG.
- Actions such as `fit`, `evaluate`, `write` trigger execution.

Performance Strategy

- GZIP files are not splittable, so one partition is created per file.
- Train and test DataFrames are cached using `MEMORY_AND_DISK`.
- Shuffle partitions tuned for single-machine execution.
- Driver memory: 6 GB.

Spark Data Flow

Raw NOAA Files $\rightarrow$ Parse and Clean $\rightarrow$ Feature Engineering $\rightarrow$ Train / Test Split $\rightarrow$ Model Training $\rightarrow$ Evaluation

Scala Implementation

```scala
// Batch layer: parse NOAA fixed-width records
val parsed = rawLines.flatMap { line => Try {
  val wind = if (windRaw == "9999") None
             else Some(windRaw.toDouble / 10.0)

  val temp = if (tempRaw == "+9999") None
             else Some(tempRaw.toDouble / 10.0)

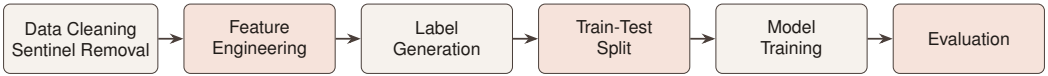
  (wind, temp)
}.toOption }.toDF("wind_speed", "temperature")

// Create proxy fire-risk label
val labeled = parsed
  .filter($"temperature".isNotNull &&
          $"wind_speed".isNotNull)
  .withColumn("fire_risk",
    when($"temperature" > 28 &&
         $"wind_speed" > 5, 1.0)
    .otherwise(0.0))
```

Project Structure

```
bda-scala/
|- build.sbt
|- src/main/scala/
| - BDAProject.scala
| - BDAProjectTrain.scala
| - BDAProjectCompare.scala
| - BDAProjectStream.scala
|- weather_generator.py
|- social_generator.py
|- dashboard_server.py
|- static/
|- docker/
‘- docs/
```

Machine Learning Pipeline



Features and Label

- Features: temperature and wind speed.
- Label: `fire_risk = 1` if temperature $> 28^{\circ}$C and wind speed > 5 m/s.
- Split: 80% training, 20% testing, seed 42.

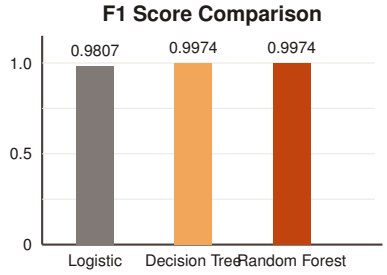
Models and Tuning

- Logistic Regression.
- Decision Tree.
- Random Forest.
- Random Forest grid: trees $\in \{20, 50, 100\}$, depth $\in \{3, 5, 10\}$.
- Best configuration: 20 trees, depth 3.

Results and Business Insight

Model Comparison

Model	F1	Precision	Recall	Time (s)
Logistic Regression	0.9807	0.9799	0.9826	33.4
Decision Tree	0.9974	0.9976	0.9973	25.9
Random Forest	0.9974	0.9976	0.9973	21.1



Full-Data Random Forest

F1: **0.9975** • Precision: 0.9977 • Recall: 0.9974

Business Insight

An alert is generated only when risky weather and a social fire report match in the same geographic grid and time window. This dual-evidence approach reduces false alarms and provides more reliable early warning.

Conclusion and Future Work

Key Achievements

- End-to-end HDFS and Spark big-data pipeline.
- Batch processing of 22.1 GB weather archive.
- Real-time verification using weather and social evidence.
- Missing-data-resilient streaming pipeline.
- Live Flask and Leaflet map dashboard.

Future Work

- Validate alerts using NASA FIRMS fire records.
- Add humidity, vegetation and terrain features.
- Load the trained ML model inside the streaming pipeline.
- Use real social-media feeds with NLP-based geotagging.
- Deploy on a multi-node YARN or Kubernetes cluster.

Limitations

- Fire-risk label is a proxy rule, not real fire ground truth.
- Spark currently runs in `local[*]` mode.
- Live weather and social feeds are simulated.

References and Code Repository

Code Repository

GitHub: https://github.com/AmithaMahesh/Wildfire-Risk-Prediction-System

A. Smith, N. Lott, and R. Vose, "The Integrated Surface Database," *Bulletin of the American Meteorological Society*, vol. 92, no. 6, 2011.

K. Shvachko et al., "The Hadoop Distributed File System," *IEEE MSST*, 2010.

M. Zaharia et al., "Resilient Distributed Datasets," *NSDI*, 2012.

M. Armbrust et al., "Spark SQL: Relational Data Processing in Spark," *SIGMOD*, 2015.

M. Armbrust et al., "Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark," *SIGMOD*, 2018.

X. Meng et al., "MLlib: Machine Learning in Apache Spark," *JMLR*, vol. 17, no. 34, 2016.

L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, 2001.

N. Marz and J. Warren, *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*, Manning, 2015.